

GENETIC ALGORITHM

Code :

```
import random
```

```
POP_SIZE = 100
```

```
CHROM_LENGTH = 20
```

```
MAX_GEN = 50
```

```
MUTATION_RATE = 0.05
```

```
class Individual:
```

```
    def __init__(self, genes=None):
```

```
        if genes is None:
```

```
            self.genes = [random.randint(0, 1) for _ in range(CHROM_LENGTH)]
```

```
        else:
```

```
            self.genes = genes
```

```
            self.fitness = self.evaluate_fitness()
```

```
    def evaluate_fitness(self):
```

```
        return sum(self.genes)
```

```
    def __str__(self):
```

```
        return f"Fitness: {self.fitness}"
```

```
def initialize_population():
```

```
    return [Individual() for _ in range(POP_SIZE)]
```

```
def select_parent(population):
```

```
    i, j = random.sample(range(POP_SIZE), 2)
```

```
    return population[i] if population[i].fitness > population[j].fitness else population[j]
```

```
def crossover(parent1, parent2):
```

```
    point = random.randint(1, CHROM_LENGTH - 1)
```

```
    child_genes = parent1.genes[:point] + parent2.genes[point:]
```

```
    return Individual(child_genes)
```

```
def mutate(individual):
```

```
    for i in range(CHROM_LENGTH):
```

```
        if random.random() < MUTATION_RATE:
```

```
            individual.genes[i] = 1 - individual.genes[i]
```

GENETIC ALGORITHM

```
individual.fitness = individual.evaluate_fitness()
```

```
def get_best_individual(population):  
    return max(population, key=lambda ind: ind.fitness)
```

```
def genetic_algorithm():  
    population = initialize_population()  
    for gen in range(MAX_GEN):  
        new_population = []  
        for _ in range(POP_SIZE):  
            parent1 = select_parent(population)  
            parent2 = select_parent(population)  
            child = crossover(parent1, parent2)  
            mutate(child)  
            new_population.append(child)  
        population = new_population  
        best = get_best_individual(population)  
        print(f"Best Fitness = {best.fitness}")  
    return best
```

```
best_solution = genetic_algorithm()  
print(f"\nBest solution found:\n{best_solution}")
```

GENETIC ALGORITHM

Solution :

[illegible]